

Beyond Monads: Pattern Matching Alternatives for Result Types

Beyond Monads

Pattern Matching Alternatives for Result Types

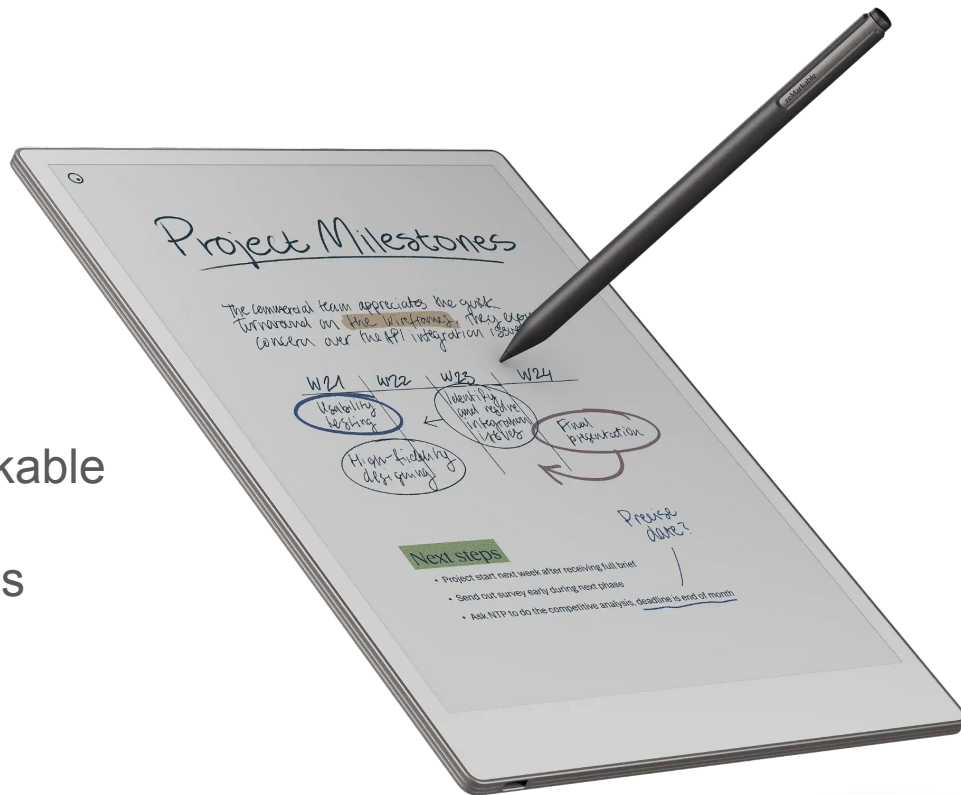
VITALY FANASKOV

About me

Vitaly Fanaskov

Principal Software Engineer at reMarkable

Work on C++ frameworks and libraries



Agenda

- Briefly about result types
- Briefly about monadic operations
- Questionable use-cases for monadic operations
- Implementing `std::visit` for result types
- Various improvements and examples
- Conclusion

Result types

What are “result types”?

- Special return types
- Contain one of two alternatives
- The alternatives can be semantically neutral or heavily biased

The result type examples

- `std::expected<T, E>`
 - T – expected result
 - E – an error
- `std::optional<T>`
 - T – contained value
- `std::variant<L, R>` (`either<L, R>`)
 - L – left alternative
 - R – right alternative

Is `std::optional` asymmetrical?

- Implementation-wise, most likely yes
- You can think of it like this: `T | nullopt_t`

Result types are sum types

expected	optional	either
$T \mid E$	$T \mid \text{nullopt_t}$	$L \mid R$

All of them can be implemented using `std::variant`

```
template <class T, class E>  
using expected = std::variant<T, E>;
```

```
template <class T>  
using optional = std::variant<T, std::nullopt_t>;
```

```
template <class L, class R>  
using either = std::variant<L, R>;
```

Working with alternatives

Let's introduce some types first

```
struct Widget
{
    std::string name{};
};
```

```
struct Error
{
    std::string reason{};
};
```

```
using Result = std::expected<Widget, Error>;
```

```
using UUID = std::string;
```

```
Result loadWidget(const UUID& id);
```

Handle contained value

```
UUID uuid = /* */;  
auto result = loadWidget(uuid);  
if (result.has_value()) {  
    // Do something with the result  
} else {  
    // Handle and error  
}
```

```
UUID uuid = /* */;  
auto result = loadWidget(uuid)  
    .and_then([](auto &&w) → Result {  
        // Do something with the result  
        // Return  
    }).or_else([](auto &&e) → Result {  
        // Handle an error  
        // Return  
    });
```

For more information



[Monadic Operations
in Modern C++: A
Practical Approach -
Vitaly Fanaskov -
CppCon 2024](#)

The slide has a dark blue background. In the top right corner, there is a green graphic element consisting of a large plus sign and the number '24'. The title 'Monadic Operations in Modern C++:' is written in a large, bold, orange font, with 'A Practical Approach' in a smaller white font below it. The speaker's name 'VITALY FANASKOV' is in orange on the right side. At the bottom left is the Cppcon logo, which includes a stylized plus sign in a circle and the text 'Cppcon The C++ Conference'. At the bottom right is the CppCon 2024 logo, featuring the year '2024' and a stylized mountain graphic with the dates 'September 15 - 20'.

Monadic operations
are the preferred
method

For some scenarios, monadic operations are redundant

- Trace or log
 - Print an object or an error directly
 - Just invoke a log function
- Pass the object further to a different type of API
 - Asynchronous API
 - Qt slot/signals
- Unit tests
 - Check if value matched
 - Assert if error

In these scenarios it's not necessary to

- Return an object of a result type
- Have limitations to a result type
- Handle the result of the entire chain

Is it possible to approach these cases differently?

Introduce more types

```
template <class L, class R>
using either = std::variant<L, R>;

struct Window { std::string name{}; };
using Result = either<Widget, Window>;

// Cannot fail
Result load(const UUID& id);
```

Let's take closer look at std::variant

```
UUID uuid = /* */;  
  
const auto widgetOrWindow = load(uuid);  
  
if (std::holds_alternative<Window>(widgetOrWindow)) {  
    const auto &window = std::get<Window>(widgetOrWindow);  
    // Do something with the window  
} else {  
    const auto &widget = std::get<Widget>(widgetOrWindow);  
    // Do something with the widget  
}  
  
// It's also possible to use std::get_if
```

This approach doesn't improve the situation much

- More generic
- Explicit if-else
- Can throw an exception

std::visit as an analogue of monadic operations

// Helper type for the visitor: <https://en.cppreference.com/cpp/utility/variant/visit>

```
template<class... Ts>
```

```
struct overloads : Ts... { using Ts::operator()...; };
```

```
// ...
```

```
UUID uuid = /* */;
```

```
const auto widgetOrWindow = load(uuid);
```

```
auto visitor = overloads{
```

```
    [](const Widget& widget) { /* Do something with the widget */ },
```

```
    [](const Window& window) { /* Do something with the window */ }
```

```
};
```

```
std::visit(visitor, widgetOrWindow);
```

Pros and cons of using `std::visit`

- + No explicit if-else
- + No need to return an object
- + Preserves value category and constness
- Bizarre API
- Requires extra helpers
- Alternatives must be different
- All alternatives should be covered

The std::visit doesn't work with other result types

```
UUID uuid = /* */;  
std::expected<Widget, Error> result = loadWidget(uuid);  
  
auto visitor = overloads{  
    [](const Widget& widget) { /* Do something with the widget */ },  
    [](const Error& error) { /* Do something with the error */ }  
};  
  
std::visit(visitor, result);
```

error: no matching function for call to 'visit(...)'

```
155 |         std::visit(visitor, result);  
    |         ~~~~~^~~~~~
```

Is it possible to hack it? (Don't do this!)

```
template <class T>
inline constexpr bool is_expected = false;
```

```
template <class T, class E>
inline constexpr bool is_expected<std::expected<T, E>> = true;
```

```
template <class T>
concept IsExpected = is_expected<std::remove_cvref_t<T>>;
```

```
namespace std
{
    template <class Visitor, IsExpected ...Expected>
    constexpr decltype(auto) visit(Visitor&& visitor, Expected&& ...expected) noexcept;
}
```

Naive implementation of std::visit for std::expected

```
template <class Visitor, IsExpected ...Expected>
constexpr decltype(auto) visit(Visitor&& visitor, Expected&& ...expected) noexcept
{
    ([&visitor]<class Exp>(Exp &&exp) {
        if (exp.has_value()) {
            return std::invoke(std::forward<Visitor>(visitor),
                               std::forward<Exp>(exp).value());
        } else {
            return std::invoke(std::forward<Visitor>(visitor),
                               std::forward<Exp>(exp).error());
        }
    }(std::forward<Expected>(expected)), ...);
}
```

Use std::visit for std::expected

```
auto visitor = overloads{  
    [](const Widget&) { std::println("Widget"); },  
    [](const Error&) { std::println("Error"); }  
};
```

```
std::visit(visitor, Result{Widget{}}, Result{std::unexpected{Error{}}});
```

```
// Prints:  
//   Widget  
//   Error
```

Use std::visit for std::expected – more generic

```
template <
struct std::formatter<Widget> : private std::formatter<std::string>;

template <
struct std::formatter<Error> : private std::formatter<std::string>;

// ...

std::visit([](auto &o) { std::println("{} ", o); },
    Result{Widget{}} ,
    Result{std::unexpected{Error{}}})
);
```

Adopting `std::visit`

Naive implementation issues



In namespace std



Doesn't handle void



Doesn't handle std::optional



Follows std::visit API



Can return an object



Not restricted

Major issues

Address major issues

- In namespace std
- Doesn't handle void
- Doesn't handle std::optional

Implement `fl::visit` for `either-like`

Introduce a simple either

```
namespace detail
{
    template <class T>
    concept ValidEitherAlternative =
        !std::same_as<std::remove_cvref_t<T>, std::monostate> &&
        !std::is_void_v<std::remove_cvref_t<T>>;
}

template <detail::ValidEitherAlternative L, detail::ValidEitherAlternative R>
class either : public std::variant<L, R>
{
public:
    using left_t = std::remove_cvref_t<L>;
    using right_t = std::remove_cvref_t<R>;

    using std::variant<L, R>::variant;
};
```

Implement fl::visit for either-like

```
template <class Visitor, detail::IsEither ...Either>
constexpr decltype(auto) visit(Visitor&& visitor, Either&& ...either) noexcept
{
    ([&visitor]<class E>(E &&e) {
        if (std::holds_alternative<detail::either_left_t<E>>(e)) {
            return std::invoke(std::forward<Visitor>(visitor),
                               std::get<detail::either_left_t<E>>(std::forward<E>(e)));
        } else {
            return std::invoke(std::forward<Visitor>(visitor),
                               std::get<detail::either_right_t<E>>(std::forward<E>(e)));
        }
    })(std::forward<Either>(either), ...);
}
```

Implement `fl::visit` for `std::expected`

Take it out of the namespace std

```
namespace fl
{
    namespace detail
    {
        // All is-expected-related
    }

    template <class Visitor, detail::IsExpected ...Expected>
    constexpr decltype(auto) visit(Visitor&& visitor, Expected&& ...expected) noexcept
    {
        // Remains the same
    }
}
```

Handle cases of void value type for expected

```
namespace detail
{
    template <class Expected>
    concept HasVoidValueType =
        std::is_void_v<typename std::remove_cvref_t<Expected>::value_type>;

    template <class Visitor, class Expected>
    constexpr decltype(auto) invokeForValue(Visitor&& visitor, Expected&& expected) noexcept
    {
        if constexpr (HasVoidValueType<Expected>) {
            return std::invoke(std::forward<Visitor>(visitor));
        } else {
            return std::invoke(std::forward<Visitor>(visitor),
                               std::forward<Expected>(expected).value());
        }
    }
}
```

Add void handling fl::visit

```
template <class Visitor, detail::IsExpected ...Expected>
constexpr decltype(auto) visit(Visitor&& visitor, Expected&& ...expected) noexcept
{
    ([&visitor]<class Exp>(Exp &&exp) {
        if (exp.has_value()) {
            return detail::invokeForValue(std::forward<Visitor>(visitor),
                std::forward<Exp>(exp));
        } else {
            return std::invoke(std::forward<Visitor>(visitor),
                std::forward<Exp>(exp).error());
        }
    }(std::forward<Expected>(expected)), ...);
}
```

Test it works for void

```
using Result = std::expected<void, std::string>;

auto visitor = overloads{
    [] { std::println("void"); },
    [](const std::string&) { std::println("std::string"); }
};

fl::visit(visitor,
    Result{},
    Result{std::unexpected{std::string{}}})
);

// Prints:
//      void
//      std::string
```

Handle the same alternatives

```
using Result = std::expected<int, int>;
```

```
auto visitor = overloads{  
    // It's impossible:  
    //      - to have two functions with the same signature  
    //      - handle T and E separately  
    [](int i) { std::println("int: {}", i); }  
};
```

```
fl::visit(visitor, Result{1}, Result{std::unexpected{2}});
```

```
// Prints:
```

```
//      int: 1
```

```
//      int: 2
```

What to do with the same types for alternatives?

- Restrict
- Leave the situation as is

Implement `fl::visit` for `std::optional`

Implement fl::visit for std::optional

```
template <class Visitor, detail::IsOptional ...Optional>
constexpr decltype(auto) visit(Visitor&& visitor, Optional&& ...optional) noexcept
{
    ([&visitor]<class Opt>(Opt &&opt) {
        if (opt.has_value()) {
            return std::invoke(
                std::forward<Visitor>(visitor), std::forward<Opt>(opt).value());
        } else {
            return std::invoke(std::forward<Visitor>(visitor));
        }
    })(std::forward<Optional>(optional), ...);
}
```

Test fl::visit for std::optional

```
using Result = std::optional<int>;

auto visitor = overloads{
    [] { std::println("nullopt"); },
    [](int) { std::println("int"); }
};

fl::visit(visitor, Result{}, Result{42});

// Prints:
//      nullopt
//      int
```

Minor issues

Address some of potential issues

- Follows `std::visit` API
- Can return an object
- Too generic

Parameters order of fl::visit

```
visit(Visitor&& visitor,  
      Expected&& ...expected)
```

```
visit(Expected&& expected,  
      Visitor&& ...visitor)
```

- One object to inspect
- Zero or many visitors (should be one or two for optional and expected)
- More natural order of parameters

Change the parameter order of fl::visit (either)

```
template <detail::IsEither Either, class ...Visitor>
constexpr decltype(auto) visit(Either &&either, Visitor&& ...visitor) noexcept
{
    auto v = overloads{std::forward<Visitor>(visitor)...};
    return visit(v, std::forward<Either>(either));
}
```

Change the parameter order of fl::visit (expected)

```
template <detail::IsExpected Expected, class ...Visitor>
constexpr decltype(auto) visit(Expected &&expected, Visitor&& ...visitor) noexcept
{
    auto v = overloads{std::forward<Visitor>(visitor)...};
    return visit(v, std::forward<Expected>(expected));
}
```

Change the parameter order of fl::visit (optional)

```
template <detail::IsOptional Optional, class ...Visitor>
constexpr decltype(auto) visit(Optional &&optional, Visitor&& ...visitor) noexcept
{
    auto v = overloads{std::forward<Visitor>(visitor)...};
    return visit(v, std::forward<Optional>(optional));
}
```

Introduce new constraints

```
// For std::expected only to avoid ambiguity (optional constraint)
```

```
template <class E>
```

```
concept ValueAndErrorTypeSame = std::same_as<exp_val_t<E>, exp_err_t<E>>;
```

```
template <class ...Visitors>
```

```
concept OneOrTwoVisitors = sizeof...(Visitors) == 1 || sizeof...(Visitors) == 2;
```

Introduce new constraints for visitors

```
template <class F, class R>  
inline constexpr bool is_invocable_with = false;
```

```
template <class F, IsEither E>  
inline constexpr bool is_invocable_with<F, E> =  
    std::invocable<F, either_left_t<E>> && std::invocable<F, either_right_t<E>>;
```

```
template <class F, IsExpected E>  
inline constexpr bool is_invocable_with<F, E> =  
    (HasVoidValueType<E> && std::invocable<F> || std::invocable<F, exp_val_t<E>>) &&  
    std::invocable<F, exp_err_t<E>>;
```

```
template <class F, IsOptional O>  
inline constexpr bool is_invocable_with<F, O> =  
    std::invocable<F> && std::invocable<F, opt_val_t<O>>;
```

Introduce a concept for visitors

```
template <class Overloads, class R>  
concept IsInvocableWith = is_invocable_with<Overloads, R>;
```

Use new constraints in fl::visit for either

```
static_assert(detail::OneOrTwoVisitors<Visitor...>,  
              "One or two visitors expected");
```

```
auto v = overloads{std::forward<Visitor>(visitor)...};
```

```
static_assert(detail::IsInvocableWith<decltype(v), Either>,  
              "Visitor must be invocable with both alternatives");
```

Use new constraints in fl::visit for expected

```
static_assert(!detail::ValueAndErrorTypeSame<Expected>,  
              "Value type and error type must be different");  
static_assert(detail::OneOrTwoVisitors<Visitor...>,  
              "One or two visitors expected");  
  
auto v = overloads{std::forward<Visitor>(visitor)...};  
  
static_assert(detail::IsInvocableWith<decltype(v), Expected>,  
              "Visitor must be invocable with both alternatives");
```

Use new constraints in fl::visit for optional

```
static_assert(detail::OneOrTwoVisitors<Visitor...>,  
              "One or two visitors expected");
```

```
auto v = overloads{std::forward<Visitor>(visitor)...};
```

```
static_assert(detail::IsInvocableWith<decltype(v), Optional>,  
              "Visitor must be invocable with both alternatives");
```

It's possible to have a more universal interface

```
template <class R>
concept IsResultType = IsEither<R> || IsExpected<R> || IsOptional<R>;

template <detail::IsResultType Result, class ...Visitor>
constexpr decltype(auto) visit(Result &&result, Visitor&& ...visitor) noexcept
{
    static_assert(!detail::ValueAndErrorTypeSame<Result>, "Value type and error type must be different");
    static_assert(detail::OneOrTwoVisitors<Visitor...>, "One or two visitors expected");

    auto v = overloads{std::forward<Visitor>(visitor)...};

    static_assert(detail::IsInvocableWith<decltype(v), Result>,
        "Visitor must be invocable with both alternatives");

    return visit(v, std::forward<Result>(result));
}
```

Addressed minor issues

- ~~Follows std::visit API~~
- Can return an object – we don't need it, but it's fine if we keep it
- ~~Not restricted~~

What is missing?

- “detail::ValueAndErrorTypeSame” should be implemented for all result types
- We need a better version of “overloads”
 - visit(obj, leftFunc, rightFunc)
 - visit(obj, f) { return visit(f, f); }

Final implementation for either: part 1

```
template <class Self, class LeftFunc, class RightFunc>
    requires(/* ... */)
[[nodiscard]] constexpr decltype(auto) visit(this Self &&self, LeftFunc lf, RightFunc rf)
    noexcept(/* ... */)
{
    using fwd_left_t = detail::either_::ForwardAlternativeT<Self, left_t>;
    using fwd_right_t = detail::either_::ForwardAlternativeT<Self, right_t>;

    static_assert(detail::either_::InvocableV<LeftFunc, fwd_left_t>, "...");
    static_assert(detail::either_::InvocableV<RightFunc, fwd_right_t>, "...");
    static_assert(detail::either_::SameReturnType<
        LeftFunc, fwd_left_t, RightFunc, fwd_right_t>, "...");

    // ...
```

Final implementation for either: part 2

```
template <class Self, class LeftFunc, class RightFunc>
    requires(/* ... */)
[[nodiscard]] constexpr decltype(auto) visit(this Self &&self, LeftFunc lf, RightFunc rf)
    noexcept(/* ... */)
{
    // ...
    if (self.is_left()) {
        if constexpr (detail::either_::IsVoid<left_t>) { return std::invoke(lf); }
        else { return std::invoke(lf, std::forward<Self>(self).left()); }
    } else {
        if constexpr (detail::either_::IsVoid<right_t>) { return std::invoke(rf); }
        else { return std::invoke(rf, std::forward<Self>(self).right()); }
    }
}
```

Examples

Log/trace

```
auto result = o
    .and_then([](const ui::Widget& w)
        → Result {
            std::println("Widget: {}", w);
            return w;
        })
    .or_else([](const ui::Error& e)
        → Result {
            std::println("Error: {}", e);
            return std::unexpected{e};
        });
```

```
fl::visit(o,
    [](const ui::Widget& w) {
        std::println("Widget: {}", w); },
    [](const ui::Error& e) {
        std::println("Error: {}", e); }
    );
```

Often objects are formattable

```
void trace(const std::formattable<char> auto& obj)
{
    std::println("{} ", obj);
}
```

```
fl::visit(o, [] (const auto& w) { trace(w); });
```

Qt signals

```
auto result = loadWidget(id)
    .and_then([](const Widget& w) → Result {
        Q_EMIT widgetLoaded(w.name);
        return w;
    })
    .or_else([](const Error& e) → Result {
        Q_EMIT widgetLoadingFailure(e);
        return std::unexpected{e};
    });
```

```
fl::visit(loadWidget(id),
    [](const Widget& w) {
        Q_EMIT widgetLoaded(w.name);
    },
    [](const Error& e) {
        Q_EMIT widgetLoadingFailure(e);
    }
);
```

Tests

```
auto result = loadWidget(id)
    .and_then([](const Widget& w) → Result {
        REQUIRE(w.name == "widget1");
        return w;
    })
    .or_else([](const Error& e) → Result {
        FAIL(e.reason);
        return std::unexpected{e};
    });
```

```
fl::visit(loadWidget(id),
    [](const Widget& w) {
        REQUIRE(w.name == "widget1");
    },
    [](const Error& e) {
        FAIL(e.reason);
    }
);
```

Will we get real pattern matching?

Pattern matching proposal



[Pattern Matching: match Expression](#)

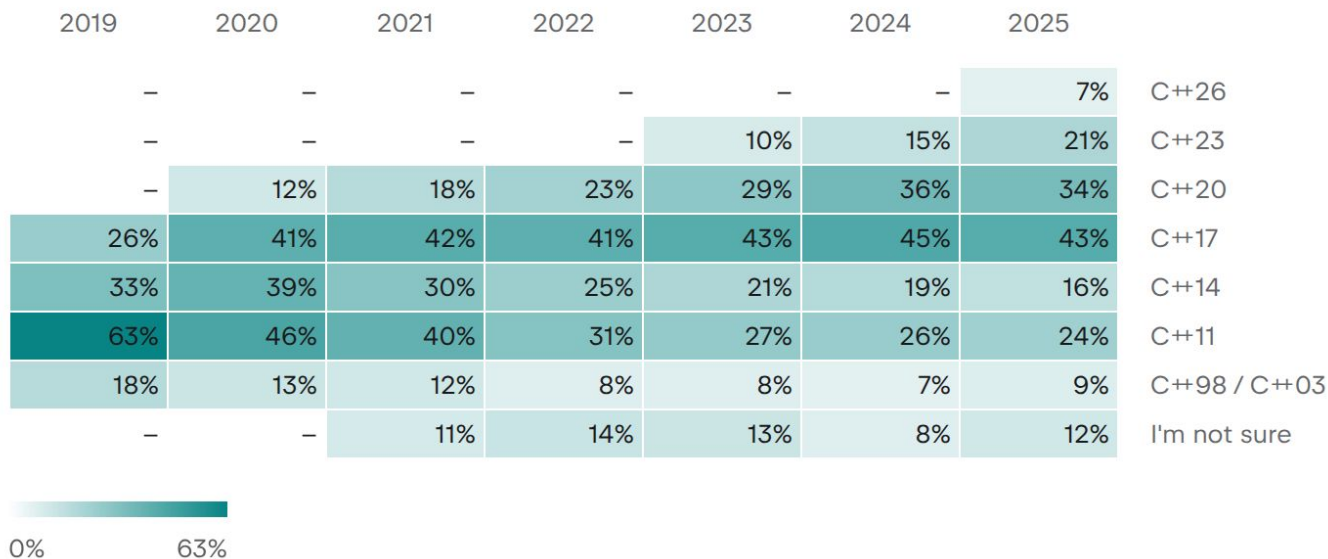
```
loadWidget(id) match {  
  Widget: let w ⇒  
    std::println("Widget: {}", w);  
  Error: let e ⇒  
    std::println("Error: {}", e);  
}
```

The pattern matching is very, very far away



[JetBrains Developer Ecosystem
Survey 2025](#)

Which C++ standards do you regularly use?



Conclusion

Conclusion

- `std::visit` can be adopted for result types
- ...better as a helper in a separate namespace
- It helps to create simpler and more explicit code when we don't need to use a contained object after
- Regular pattern matching is the best alternative
- ...however, many of us will most likely not be able to use it anytime soon

Thank you!

Slides & Code



[Slides](#)



[Code](#)